

```
class OrderService {  
    private PaymentService paymentService = new PaymentService(); // tightly coupled  
}
```

3 Major Problems with this code:

1. Tight coupling
2. Hard to test
3. Scattered Object Creation

Introduction To Dependency Injection

DI is a design pattern where an object receives its dependencies from the outside, instead of creating them itself.

```
// BAD: Tight coupling
public class OrderService {
    private EmailService emailService = new EmailService();
}

// GOOD: Loose coupling via DI
public class OrderService {
    private final EmailService emailService;

    public OrderService(EmailService emailService) {
        this.emailService = emailService;
    }
}
```

Types of Dependency Injection

Type	Annotation	Recommended?
Constructor	@Autowired (optional)	YES – Best
Setter	@Autowired	OK for optional
Field	@Autowired	Avoid in production

How Spring Resolves Dependencies

1. By Type → EmailService
2. If multiple beans → use name or @Qualifier
3. If still ambiguous → use @Primary

```
@Bean
@Qualifier("smtp")
public EmailService smtpEmailService() { ... }

@Bean
@Qualifier("sendgrid")
public EmailService sendgridEmailService() { ... }
```

```
public OrderService(@Qualifier("sendgrid") EmailService emailService) { ... }
```

Dependency Injection Misc.

Get all bean instances:

```
@Autowired  
private Map<String, PaymentGateway> gateways; // bean name → instance
```

Optional Beans

```
public OrderService(Optional<AnalyticsService> analytics) { ... }
```

```
@Autowired(required = false)  
private BackupService backupService; // null if not present
```

1. Interface ka object create nahi hota hence uske uper @Component lagana is useless
2. @Primary - If ek type ke multiple bean defined hai then jiske uper yeh usko preference
3. Field Injection ki wajah se field mutable ho sakte hai *
4. Constructor DI is preferred ; We can make the field final in C-DI which makes it immutable
5. Final Variable cannot be initialized after the constructor LOC
6. Conditional based resolution ke case me non-matching bean create hi nahi honge (different from use nahi honge)
7. Bean usually create hoke container me padh rhte then on call/requirement use hote hai
8. Exclusive - Ek time per dono nahi
9. name & value field of Conditional Annotation same chiz hi alag tarike se represent krte hence should be exclusive
10. Property Source - Environment Variables , Property File , ...
11. Environment Variable > Property File ; ConditionalOnProperty works for both of them but style property wala
12. To have/use all bean of particular type then use Map<> or Set
13. Spring ke autowiring contract me yah pahle se defined hai ki Map<String, T> ka matlab hota hai key=BeanName and Value=BeanInstanceOfTypeT
14. Spring automatically default bean name generate krta h jisme class name ka first letter lowercase karke bana deta
15. If some bean is not created even then also you want to have hold over that bean then use optional bean approach

1. Tight coupling resolution - DI
2. Multiple Bean of same type conflict
3. Primary
4. Field DI
5. Constructor DI
6. Qualifier
7. Conditional - property file & env variables
8. Multiple bean of same type usage - Map<String, T>
9. Optional Bean